

Backend Database Schema Document

Project: CareerGuid AI / SkillSight

Database: Supabase PostgreSQL

Storage: Supabase Storage

Authentication: Supabase Auth + JWT

Cache/Queue: Redis + BullMQ

1. Database Design Principles

This schema is designed for:

- Resume-first onboarding
- AI-based profile auto-fill
- Skill gap analysis
- RAG-based roadmap generation
- Weekly reminder emails
- ATS resume builder
- Payment/upgrade flow
- Admin management
- Supabase Row Level Security
- Future scalability

Main rule:

Every user-owned table should have user_id UUID REFERENCES auth.users(id) so ownership can be enforced using Supabase RLS.

2. Required PostgreSQL Extensions

```
create extension if not exists "uuid-ossf";  
create extension if not exists "pgcrypto";  
create extension if not exists "vector";
```

vector is needed for Supabase pgvector-based RAG search.

3. Enum Types

```
create type user_role as enum ('user', 'admin');

create type work_preference as enum ('remote', 'hybrid', 'onsite');

create type parsing_status as enum (
    'pending',
    'processing',
    'completed',
    'failed'
);

create type task_status as enum (
    'pending',
    'completed',
    'skipped',
    'overdue'
);

create type reminder_type as enum (
    'weekly_pending_task',
    'inactive_user',
    'motivational',
    'roadmap_due'
);

create type email_status as enum (
    'queued',
    'sent',
    'failed'
);

create type subscription_status as enum (
    'free',
    'active',
    'cancelled',
    'expired',
    'payment_failed'
);

create type ai_job_status as enum (
    'queued',
    'processing',
    'completed',
```

```
'failed'  
);
```

4. Core User Tables

4.1 profiles

This table stores user profile data linked to Supabase Auth.

Supabase already stores auth data in:

auth.users

Your custom user data goes into:

public.profiles

Table

```
create table profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  
  full_name text,  
  email text not null,  
  phone text,  
  current_city text,  
  
  education text,  
  work_experience text,  
  skills text[] default '{}',  
  projects jsonb default '[]',  
  
  linkedin_url text,  
  github_url text,  
  portfolio_url text,  
  
  target_job_role text,  
  preferred_location text,  
  work_preference work_preference,  
  
  expected_salary text,  
  notice_period text,  
  career_goal text,  
  
  role user_role default 'user',
```

```

onboarding_completed boolean default false,
email_verified boolean default false,

created_at timestampz default now(),
updated_at timestampz default now()
);

```

Important Columns

Column	Purpose
id	Same as Supabase Auth user ID
role	User/admin permission control
onboarding_completed	Decides whether user should go to onboarding or dashboard
skills	Extracted or manually added skills
projects	Project data from resume/manual input

Indexes

```

create index idx_profiles_email on profiles(email);
create index idx_profiles_role on profiles(role);
create index idx_profiles_onboarding on profiles(onboarding_completed);

```

4.2 profile_field_sources

This table tracks whether profile fields came from resume, manual input, or AI extraction. It is useful for smart onboarding.

Table

```

create table profile_field_sources (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references auth.users(id) on delete cascade,

  field_name text not null,
  field_value text,
  source text not null check (source in ('resume', 'manual', 'ai', 'system')),
  confidence_score numeric(5,2),

```

```

is_review_required boolean default false,

created_at timestamptz default now(),
updated_at timestamptz default now(),

unique(user_id, field_name)
);

```

Example

Field	Source	Confidence
full_name	resume	98.50
phone	resume	92.00
target_job_role	manual	100.00

Indexes

```

create index idx_profile_field_sources_user_id on profile_field_sources(user_id)
create index idx_profile_field_sources_source on profile_field_sources(source);

```

5. Resume Tables

5.1 resumes

Stores resume file metadata and AI-extracted data.

Table

```

create table resumes (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references auth.users(id) on delete cascade,

  file_name text not null,
  file_url text not null,
  storage_path text not null,
  file_type text not null,
  file_size integer not null,

  extracted_text text,
  extracted_data jsonb default '{}',

```

```

extracted_skills text[] default '{}',

parsing_status parsing_status default 'pending',
parsing_error text,

is_active boolean default true,

created_at timestampz default now(),
updated_at timestampz default now()
);

```

Important Notes

- Actual file is stored in Supabase Storage.
- This table stores metadata and parsing result.
- is_active = true means current active resume.

Indexes

```

create index idx_resumes_user_id on resumes(user_id);
create index idx_resumes_status on resumes(parsing_status);
create index idx_resumes_active on resumes(user_id, is_active);

```

6. Job Role and Skills Tables

6.1 job_roles

Stores available job roles.

Table

```

create table job_roles (
  id uuid primary key default gen_random_uuid(),

  role_name text not null unique,
  role_slug text not null unique,
  role_description text,
  category text,

  is_active boolean default true,

  created_at timestampz default now(),

```

```
    updated_at timestamptz default now()
);
```

Example Rows

Backend Developer
 Frontend Developer
 Full Stack Developer
 Data Analyst
 AI/ML Engineer
 Cloud Engineer
 DevOps Engineer

Indexes

```
create index idx_job_roles_slug on job_roles(role_slug);
create index idx_job_roles_category on job_roles(category);
create index idx_job_roles_active on job_roles(is_active);
```

6.2 skills

Master skill table.

Table

```
create table skills (
  id uuid primary key default gen_random_uuid(),

  skill_name text not null unique,
  normalized_name text not null unique,
  category text,
  description text,

  created_at timestamptz default now()
);
```

Example

skill_name	normalized_name	category
Node.js	nodejs	Backend
Express.js	expressjs	Backend
MongoDB	mongodb	Database

skill_name	normalized_name	category
Supabase	supabase	Database

Indexes

```
create index idx_skills_normalized_name on skills(normalized_name);
create index idx_skills_category on skills(category);
```

6.3 skill_aliases

Used for matching similar skill names.

Example:

```
JS -> JavaScript
Node -> Node.js
Postgres -> PostgreSQL
```

Table

```
create table skill_aliases (
  id uuid primary key default gen_random_uuid(),

  skill_id uuid not null references skills(id) on delete cascade,
  alias_name text not null,
  normalized_alias text not null,

  created_at timestamptz default now(),

  unique(skill_id, normalized_alias)
);
```

Indexes

```
create index idx_skill_aliases_skill_id on skill_aliases(skill_id);
create index idx_skill_aliases_alias on skill_aliases(normalized_alias);
```

6.4 role_skills

Maps job roles to required skills.

Table

```
create table role_skills (  
  id uuid primary key default gen_random_uuid(),  
  
  role_id uuid not null references job_roles(id) on delete cascade,  
  skill_id uuid not null references skills(id) on delete cascade,  
  
  priority text not null check (priority in ('must_have', 'good_to_have', 'optional'),  
  skill_level text check (skill_level in ('beginner', 'intermediate', 'advanced'),  
  
  weight integer default 1,  
  
  created_at timestamptz default now(),  
  
  unique(role_id, skill_id)  
);
```

Indexes

```
create index idx_role_skills_role_id on role_skills(role_id);  
create index idx_role_skills_skill_id on role_skills(skill_id);  
create index idx_role_skills_priority on role_skills(priority);
```

7. Skill Gap Analysis Tables

7.1 skill_analyses

Stores one skill gap analysis result.

Table

```
create table skill_analyses (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null references auth.users(id) on delete cascade,  
  resume_id uuid references resumes(id) on delete set null,  
  role_id uuid not null references job_roles(id) on delete cascade,  
  
  current_skills text[] default '{}',  
  missing_skills text[] default '{}',  
  matched_skills text[] default '{}',  
  recommended_skills text[] default '{}',
```

```

match_score integer check (match_score >= 0 and match_score <= 100),
analysis_result jsonb default '{}',
created_at timestamptz default now()
);

```

Indexes

```

create index idx_skill_analyses_user_id on skill_analyses(user_id);
create index idx_skill_analyses_role_id on skill_analyses(role_id);
create index idx_skill_analyses_created_at on skill_analyses(created_at desc);

```

7.2 skill_analysis_items

Detailed skill-level analysis.

Table

```

create table skill_analysis_items (
  id uuid primary key default gen_random_uuid(),

  analysis_id uuid not null references skill_analyses(id) on delete cascade,
  skill_id uuid references skills(id) on delete set null,

  skill_name text not null,
  status text not null check (status in ('matched', 'missing', 'recommended')),
  priority text check (priority in ('high', 'medium', 'low')),
  reason text,

  created_at timestamptz default now()
);

```

Indexes

```

create index idx_skill_analysis_items_analysis_id on skill_analysis_items(analysis_id);
create index idx_skill_analysis_items_status on skill_analysis_items(status);

```

8. RAG Knowledge Base Tables

8.1 knowledge_base_documents

Stores content used for RAG.

Table

```
create table knowledge_base_documents (  
  id uuid primary key default gen_random_uuid(),  
  
  title text not null,  
  category text not null,  
  source_url text,  
  content text not null,  
  metadata jsonb default '{}',  
  
  embedding vector(1536),  
  
  is_active boolean default true,  
  
  created_by uuid references auth.users(id) on delete set null,  
  
  created_at timestampz default now(),  
  updated_at timestampz default now()  
);
```

Use vector(1536) if using OpenAI-style embeddings. If another embedding model is used, change dimension accordingly.

Indexes

```
create index idx_knowledge_base_category on knowledge_base_documents(category);  
create index idx_knowledge_base_active on knowledge_base_documents(is_active);  
  
create index idx_knowledge_base_embedding  
on knowledge_base_documents  
using ivfflat (embedding vector_cosine_ops)  
with (lists = 100);
```

8.2 rag_queries

Stores RAG query logs for debugging and improvement.

Table

```
create table rag_queries (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid references auth.users(id) on delete set null,  
  
  query_text text not null,  
  retrieved_document_ids uuid[] default '{}',  
  response_summary text,  
  model_used text,  
  
  created_at timestampz default now()  
);
```

Indexes

```
create index idx_rag_queries_user_id on rag_queries(user_id);  
create index idx_rag_queries_created_at on rag_queries(created_at desc);
```

9. Roadmap Tables

9.1 roadmaps

Stores AI-generated roadmap.

Table

```
create table roadmaps (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null references auth.users(id) on delete cascade,  
  skill_analysis_id uuid references skill_analyses(id) on delete set null,  
  role_id uuid references job_roles(id) on delete set null,  
  
  title text not null,  
  description text,  
  duration_weeks integer,  
  
  progress_percentage integer default 0 check (  
    progress_percentage >= 0 and progress_percentage <= 100  
  ),
```

```

generated_by text default 'rag_genai',
ai_response jsonb default '{}',

is_active boolean default true,

created_at timestampz default now(),
updated_at timestampz default now()
);

```

Indexes

```

create index idx_roadmaps_user_id on roadmaps(user_id);
create index idx_roadmaps_role_id on roadmaps(role_id);
create index idx_roadmaps_active on roadmaps(user_id, is_active);

```

9.2 roadmap_weeks

Stores week-wise roadmap sections.

Table

```

create table roadmap_weeks (
  id uuid primary key default gen_random_uuid(),

  roadmap_id uuid not null references roadmaps(id) on delete cascade,

  week_number integer not null,
  title text not null,
  description text,
  start_date date,
  due_date date,

  status task_status default 'pending',

  created_at timestampz default now(),
  updated_at timestampz default now(),

  unique(roadmap_id, week_number)
);

```

Indexes

```

create index idx_roadmap_weeks_roadmap_id on roadmap_weeks(roadmap_id);
create index idx_roadmap_weeks_due_date on roadmap_weeks(due_date);

```

9.3 roadmap_tasks

Stores individual tasks inside each week.

Table

```
create table roadmap_tasks (  
  id uuid primary key default gen_random_uuid(),  
  
  roadmap_id uuid not null references roadmaps(id) on delete cascade,  
  week_id uuid not null references roadmap_weeks(id) on delete cascade,  
  
  task_title text not null,  
  task_description text,  
  resource_links jsonb default '[]',  
  
  status task_status default 'pending',  
  
  due_date date,  
  completed_at timestamptz,  
  
  sort_order integer default 0,  
  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

Indexes

```
create index idx_roadmap_tasks_roadmap_id on roadmap_tasks(roadmap_id);  
create index idx_roadmap_tasks_week_id on roadmap_tasks(week_id);  
create index idx_roadmap_tasks_status on roadmap_tasks(status);  
create index idx_roadmap_tasks_due_date on roadmap_tasks(due_date);
```

10. Reminder and Notification Tables

10.1 reminder_logs

Stores weekly reminder email logs.

Table

```
create table reminder_logs (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null references auth.users(id) on delete cascade,  
  roadmap_id uuid references roadmaps(id) on delete cascade,  
  week_id uuid references roadmap_weeks(id) on delete set null,  
  
  reminder_type reminder_type not null,  
  
  pending_task_count integer default 0,  
  
  email_sent boolean default false,  
  email_status email_status default 'queued',  
  email_error text,  
  
  sent_at timestampz,  
  
  created_at timestampz default now()  
);
```

Indexes

```
create index idx_reminder_logs_user_id on reminder_logs(user_id);  
create index idx_reminder_logs_roadmap_id on reminder_logs(roadmap_id);  
create index idx_reminder_logs_type on reminder_logs(reminder_type);  
create index idx_reminder_logs_sent_at on reminder_logs(sent_at desc);
```

Duplicate Reminder Protection

To avoid sending duplicate weekly reminders:

```
create unique index idx_unique_weekly_reminder  
on reminder_logs(user_id, roadmap_id, week_id, reminder_type)  
where reminder_type = 'weekly_pending_task';
```

10.2 notifications

Stores in-app notifications.

Table

```
create table notifications (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null references auth.users(id) on delete cascade,  
  
  title text not null,  
  message text not null,  
  type text not null,  
  link_url text,  
  
  is_read boolean default false,  
  
  created_at timestampz default now(),  
  read_at timestampz  
);
```

Indexes

```
create index idx_notifications_user_id on notifications(user_id);  
create index idx_notifications_is_read on notifications(user_id, is_read);  
create index idx_notifications_created_at on notifications(created_at desc);
```

10.3 email_logs

Stores all outgoing email status.

Table

```
create table email_logs (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid references auth.users(id) on delete set null,  
  
  email_to text not null,  
  email_subject text not null,  
  email_type text not null,  
  
  status email_status default 'queued',  
  provider_message_id text,  
  error_message text,  
  
  sent_at timestampz,
```



```
    created_at timestamptz default now()
);
```

Indexes

```
create index idx_email_logs_user_id on email_logs(user_id);
create index idx_email_logs_type on email_logs(email_type);
create index idx_email_logs_status on email_logs(status);
```

11. ATS Resume Builder Tables

11.1 generated_resumes

Stores generated ATS resume versions.

Table

```
create table generated_resumes (
    id uuid primary key default gen_random_uuid(),

    user_id uuid not null references auth.users(id) on delete cascade,
    source_resume_id uuid references resumes(id) on delete set null,

    target_role text not null,

    resume_content jsonb not null default '{}',
    ats_keywords text[] default '{}',
    ats_score integer check (ats_score >= 0 and ats_score <= 100),

    pdf_url text,
    docx_url text,
    pdf_storage_path text,
    docx_storage_path text,

    is_active boolean default true,

    created_at timestamptz default now(),
    updated_at timestamptz default now()
);
```

Indexes

```
create index idx_generated_resumes_user_id on generated_resumes(user_id);
create index idx_generated_resumes_target_role on generated_resumes(target_role);
create index idx_generated_resumes_active on generated_resumes(user_id, is_active);
```

11.2 resume_templates

Stores resume templates.

Table

```
create table resume_templates (
  id uuid primary key default gen_random_uuid(),

  template_name text not null,
  template_slug text not null unique,
  description text,
  preview_image_url text,

  is_premium boolean default false,
  is_active boolean default true,

  created_at timestampz default now()
);
```

Indexes

```
create index idx_resume_templates_slug on resume_templates(template_slug);
create index idx_resume_templates_premium on resume_templates(is_premium);
```

12. AI Job Tables

12.1 ai_jobs

Tracks background AI jobs.

Table

```
create table ai_jobs (
  id uuid primary key default gen_random_uuid(),
```

```

user_id uuid references auth.users(id) on delete cascade,

job_type text not null,
status ai_job_status default 'queued',

input_payload jsonb default '{}',
output_payload jsonb default '{}',

error_message text,
retry_count integer default 0,

started_at timestampz,
completed_at timestampz,

created_at timestampz default now(),
updated_at timestampz default now()
);

```

Job Types

```

resume_parsing
skill_gap_analysis
roadmap_generation
ats_resume_generation
embedding_generation

```

Indexes

```

create index idx_ai_jobs_user_id on ai_jobs(user_id);
create index idx_ai_jobs_status on ai_jobs(status);
create index idx_ai_jobs_type on ai_jobs(job_type);
create index idx_ai_jobs_created_at on ai_jobs(created_at desc);

```

13. Payment and Upgrade Tables

13.1 plans

Stores pricing plans.

Table

```
create table plans (  
  id uuid primary key default gen_random_uuid(),  
  
  plan_name text not null unique,  
  plan_slug text not null unique,  
  description text,  
  
  price_monthly numeric(10,2) default 0,  
  price_yearly numeric(10,2) default 0,  
  currency text default 'INR',  
  
  resume_upload_limit integer,  
  roadmap_generation_limit integer,  
  ats_resume_generation_limit integer,  
  ai_chat_limit integer,  
  
  is_active boolean default true,  
  
  created_at timestamptz default now()  
);
```

Example Plans

free
pro
premium

13.2 subscriptions

Stores user subscription.

Table

```
create table subscriptions (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null references auth.users(id) on delete cascade,  
  plan_id uuid not null references plans(id),  
  
  status subscription_status default 'free',  
  
  billing_cycle text check (billing_cycle in ('monthly', 'yearly', 'none')) defa
```

```

starts_at timestamptz default now(),
ends_at timestamptz,

payment_provider text,
provider_subscription_id text,

created_at timestamptz default now(),
updated_at timestamptz default now()
);

```

Indexes

```

create index idx_subscriptions_user_id on subscriptions(user_id);
create index idx_subscriptions_status on subscriptions(status);
create index idx_subscriptions_plan_id on subscriptions(plan_id);

```

13.3 payment_transactions

Stores payment history.

Table

```

create table payment_transactions (
  id uuid primary key default gen_random_uuid(),

  user_id uuid not null references auth.users(id) on delete cascade,
  subscription_id uuid references subscriptions(id) on delete set null,

  provider text not null,
  provider_payment_id text,
  provider_order_id text,

  amount numeric(10,2) not null,
  currency text default 'INR',

  status text not null check (
    status in ('created', 'paid', 'failed', 'refunded')
  ),

  raw_response jsonb default '{}',

  created_at timestamptz default now()
);

```

Indexes

```
create index idx_payment_transactions_user_id on payment_transactions(user_id);
create index idx_payment_transactions_status on payment_transactions(status);
create index idx_payment_transactions_provider_payment_id on payment_transaction
```

13.4 usage_counters

Tracks plan usage.

Table

```
create table usage_counters (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references auth.users(id) on delete cascade,
  usage_key text not null,
  usage_count integer default 0,
  period_start date not null,
  period_end date not null,
  created_at timestampz default now(),
  updated_at timestampz default now(),
  unique(user_id, usage_key, period_start, period_end)
);
```

Usage Keys

```
resume_upload
roadmap_generation
ats_resume_generation
ai_chat
```

Indexes

```
create index idx_usage_counters_user_id on usage_counters(user_id);
create index idx_usage_counters_key on usage_counters(usage_key);
```

14. Admin and Audit Tables

14.1 audit_logs

Stores important admin/user actions.

Table

```
create table audit_logs (  
  id uuid primary key default gen_random_uuid(),  
  
  actor_user_id uuid references auth.users(id) on delete set null,  
  
  action text not null,  
  entity_type text,  
  entity_id uuid,  
  
  old_data jsonb,  
  new_data jsonb,  
  
  ip_address text,  
  user_agent text,  
  
  created_at timestampz default now()  
);
```

Indexes

```
create index idx_audit_logs_actor_user_id on audit_logs(actor_user_id);  
create index idx_audit_logs_action on audit_logs(action);  
create index idx_audit_logs_created_at on audit_logs(created_at desc);
```

14.2 system_settings

Stores app-level settings.

Table

```
create table system_settings (  
  id uuid primary key default gen_random_uuid(),  
  
  setting_key text not null unique,  
  setting_value jsonb not null,
```

```
updated_by uuid references auth.users(id) on delete set null,  
updated_at timestamptz default now()  
);
```

15. Relationships Overview

```
auth.users  
├── profiles  
├── resumes  
├── skill_analyses  
├── roadmaps  
│   ├── roadmap_weeks  
│   │   └── roadmap_tasks  
│   └── reminder_logs  
├── generated_resumes  
├── notifications  
├── ai_jobs  
├── subscriptions  
├── payment_transactions  
└── usage_counters  
  
job_roles  
├── role_skills  
├── skill_analyses  
└── roadmaps  
  
skills  
├── skill_aliases  
└── role_skills  
  
knowledge_base_documents  
└── rag_queries
```

16. Authentication and Session Handling

16.1 Authentication Method

Use:

Supabase Auth

Supported methods:

Email/password login
Email verification
Forgot password
Google login, optional
JWT session
Refresh token

16.2 Signup Flow

User submits signup form
↓
Supabase Auth creates user in auth.users
↓
Trigger creates row in profiles
↓
Verification email/OTP sent
↓
User verifies email
↓
email_verified = true
↓
User goes to onboarding

16.3 Login Flow

User logs in
↓
Supabase returns access token + refresh token
↓
Frontend stores session using Supabase client
↓
Backend receives Bearer token
↓
Backend verifies JWT
↓
User accesses protected APIs

16.4 Session Rules

- Frontend should use Supabase client session management.
- Backend should never trust frontend user ID directly.
- Backend should extract `user_id` from JWT.
- Refresh token handled by Supabase client.
- Access token sent in header:

Authorization: Bearer <token>

17. Row Level Security Rules

Enable RLS on all user-owned tables.

```
alter table profiles enable row level security;
alter table profile_field_sources enable row level security;
alter table resumes enable row level security;
alter table skill_analyses enable row level security;
alter table skill_analysis_items enable row level security;
alter table roadmaps enable row level security;
alter table roadmap_weeks enable row level security;
alter table roadmap_tasks enable row level security;
alter table reminder_logs enable row level security;
alter table notifications enable row level security;
alter table generated_resumes enable row level security;
alter table subscriptions enable row level security;
alter table payment_transactions enable row level security;
alter table usage_counters enable row level security;
```

17.1 User Ownership Rule

For tables with direct `user_id`:

```
create policy "Users can view own records"
on resumes
for select
using (auth.uid() = user_id);

create policy "Users can insert own records"
on resumes
for insert
with check (auth.uid() = user_id);
```

```
create policy "Users can update own records"
on resumes
for update
using (auth.uid() = user_id);
```

```
create policy "Users can delete own records"
on resumes
for delete
using (auth.uid() = user_id);
```

Apply the same pattern to:

```
profile_field_sources
resumes
skill_analyses
roadmaps
reminder_logs
notifications
generated_resumes
subscriptions
payment_transactions
usage_counters
ai_jobs
```

17.2 Profile Policy

```
create policy "Users can view own profile"
on profiles
for select
using (auth.uid() = id);
```

```
create policy "Users can update own profile"
on profiles
for update
using (auth.uid() = id);
```

```
create policy "Users can insert own profile"
on profiles
for insert
with check (auth.uid() = id);
```

17.3 Nested Ownership Policy

Some tables do not directly store `user_id`, like `roadmap_tasks`. Ownership is checked through parent table.

Roadmap Weeks

```
create policy "Users can view own roadmap weeks"
on roadmap_weeks
for select
using (
    exists (
        select 1 from roadmaps
        where roadmaps.id = roadmap_weeks.roadmap_id
        and roadmaps.user_id = auth.uid()
    )
);
```

Roadmap Tasks

```
create policy "Users can view own roadmap tasks"
on roadmap_tasks
for select
using (
    exists (
        select 1 from roadmaps
        where roadmaps.id = roadmap_tasks.roadmap_id
        and roadmaps.user_id = auth.uid()
    )
);
```

Update task policy:

```
create policy "Users can update own roadmap tasks"
on roadmap_tasks
for update
using (
    exists (
        select 1 from roadmaps
        where roadmaps.id = roadmap_tasks.roadmap_id
        and roadmaps.user_id = auth.uid()
    )
);
```

17.4 Public Read Tables

These tables can be readable by authenticated users:

job_roles
skills
role_skills
resume_templates
plans

Policy example:

```
create policy "Authenticated users can read job roles"
on job_roles
for select
to authenticated
using (is_active = true);
```

17.5 Admin Permission Rule

Recommended: store admin role in Supabase app_metadata.

Admin check:

```
create or replace function is_admin()
returns boolean as $$
begin
    return coalesce(
        auth.jwt() -> 'app_metadata' -> 'role',
        ''
    ) = 'admin';
end;
$$ language plpgsql stable;
```

Admin policy example:

```
create policy "Admins can manage job roles"
on job_roles
for all
using (is_admin())
with check (is_admin());
```

Admin can manage:

job_roles
skills
role_skills
knowledge_base_documents
resume_templates

system_settings
audit_logs

18. Supabase Storage Buckets

18.1 Required Buckets

resumes
generated-resumes
template-previews
knowledge-base-files

18.2 Bucket Rules

resumes

- Private bucket
- User can upload only to own folder

Path format:

resumes/{user_id}/{resume_id}.pdf

generated-resumes

Path format:

generated-resumes/{user_id}/{generated_resume_id}.pdf
generated-resumes/{user_id}/{generated_resume_id}.docx

knowledge-base-files

- Admin-only upload
 - Used for RAG documents
-

19. Storage Ownership Policy

Example for resumes bucket:

```

create policy "Users can upload own resumes"
on storage.objects
for insert
to authenticated
with check (
    bucket_id = 'resumes'
    and auth.uid()::text = (storage.foldername(name))[1]
);

create policy "Users can read own resumes"
on storage.objects
for select
to authenticated
using (
    bucket_id = 'resumes'
    and auth.uid()::text = (storage.foldername(name))[1]
);

```

20. Key Backend Business Rules

20.1 Onboarding Rules

User must upload resume first unless they choose manual fallback.
 AI auto-filled fields must be editable.
 Missing required fields must be completed before onboarding is marked completed.
 onboarding_completed becomes true only after final profile submission.

20.2 Resume Rules

Allowed file types: PDF, DOCX
 Max file size: 5 MB
 Only one active resume by default
 Old resumes can remain stored but inactive
 Resume parsing is async
 Parsing failure should allow retry or manual fill

20.3 Skill Gap Rules

Skill analysis requires completed profile
 Target role is required

Skill match score must be between 0 and 100
Latest skill analysis should be shown on dashboard

20.4 Roadmap Rules

Roadmap generation requires skill analysis
Roadmap should be divided into weeks
Each week has tasks
Progress percentage is calculated from completed tasks
Only one active roadmap per target role by default

20.5 Reminder Rules

Weekly reminder is sent if current week tasks are pending
Inactive reminder is sent if user inactive for 7 days
Duplicate reminder for same week should not be sent
All reminder emails should be logged

20.6 Payment Rules

Free users have limited AI generations
Pro/Premium users have higher limits
Usage is tracked monthly
Failed payment should not unlock premium features
Cancelled subscription should downgrade after period ends

21. Recommended Triggers

21.1 Create Profile After Signup

```
create or replace function handle_new_user()  
returns trigger as $$  
begin  
  insert into public.profiles (id, email, full_name, email_verified)  
  values (  
    new.id,  
    new.email,  
    new.raw_user_meta_data ->> 'full_name',
```



```

        false
    );

    return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
after insert on auth.users
for each row
execute procedure handle_new_user();

```

21.2 Update updated_at

```

create or replace function update_updated_at_column()
returns trigger as $$
begin
    new.updated_at = now();
    return new;
end;
$$ language plpgsql;

```

Apply to tables:

```

profiles
resumes
job_roles
roadmaps
roadmap_weeks
roadmap_tasks
generated_resumes
subscriptions
usage_counters

```

Example:

```

create trigger update_profiles_updated_at
before update on profiles
for each row
execute procedure update_updated_at_column();

```

22. Recommended Dashboard Queries

22.1 Latest Resume

```
select *  
from resumes  
where user_id = auth.uid()  
and is_active = true  
order by created_at desc  
limit 1;
```

22.2 Latest Skill Analysis

```
select *  
from skill_analyses  
where user_id = auth.uid()  
order by created_at desc  
limit 1;
```

22.3 Active Roadmap With Tasks

```
select *  
from roadmaps r  
left join roadmap_weeks w on w.roadmap_id = r.id  
left join roadmap_tasks t on t.week_id = w.id  
where r.user_id = auth.uid()  
and r.is_active = true;
```

22.4 Pending Tasks

```
select *  
from roadmap_tasks t  
join roadmaps r on r.id = t.roadmap_id  
where r.user_id = auth.uid()  
and t.status = 'pending'  
order by t.due_date asc;
```

23. Final Schema Summary

Your backend schema has these main table groups:

Auth/User:

- profiles
- profile_field_sources

Resume:

- resumes

Skills:

- job_roles
- skills
- skill_aliases
- role_skills

Skill Gap:

- skill_analyses
- skill_analysis_items

RAG:

- knowledge_base_documents
- rag_queries

Roadmap:

- roadmaps
- roadmap_weeks
- roadmap_tasks

Reminder:

- reminder_logs
- notifications
- email_logs

ATS Resume:

- generated_resumes
- resume_templates

AI Jobs:

- ai_jobs

Payment:

- plans
- subscriptions
- payment_transactions

- usage_counters

Admin/System:

- audit_logs
- system_settings

This schema is ready for Supabase implementation with secure ownership, scalable roadmap tracking, RAG support, ATS resume generation, payments, reminders, and admin management.